

❖ Data Management concepts

1) Traversing: Every data structure contains the set of data elements. Traversing the data structure means visiting each element of the data structure in order to perform some specific operation like searching or sorting.

Example: If we need to calculate the average of the marks obtained by a student in 6 different subject, we need to traverse the complete array of marks and calculate the total sum, then we will divide that sum by the number of subjects i.e. 6, in order to find the average.

2) Insertion: Insertion can be defined as the process of adding the elements to the data structure at any location. If the size of data structure is n then we can only insert $n-1$ data elements into it.

3) Deletion: The process of removing an element from the data structure is called Deletion. We can delete an element from the data structure at any random location. If we try to delete an element from an empty data structure then underflow occurs.

4) Searching: The process of finding the location of an element within the data structure is called Searching. There are two algorithms to perform searching, Linear Search and Binary Search. We will discuss each one of them later in this tutorial.

5) Sorting: The process of arranging the data structure in a specific order is known as Sorting. There are many algorithms that can be used to perform sorting, for example, insertion sort, selection sort, bubble sort, etc.

6) Merging: When two lists List A and List B of size M and N respectively, of similar type of elements, clubbed or joined to produce the third list, List C of size $(M+N)$, then this process is called merging

Foundation terms of a data Structure: Data Structure is a systematic way to organize data in order to use it efficiently. Following terms are the foundation terms of a data structure.

- **Interface** – Each data structure has an interface. Interface represents the set of operations that a data structure supports. An interface only provides the list of supported operations, type of parameters they can accept and return type of these operations.
- **Implementation** – Implementation provides the internal representation of a data structure. Implementation also provides the definition of the algorithms used in the operations of the data structure.

Characteristics of a Data Structure

- **Correctness** – Data structure implementation should implement its interface correctly.
- **Time Complexity** – Running time or the execution time of operations of data structure must be as small as possible.
- **Space Complexity** – Memory usage of a data structure operation should be as little as possible.

Need for Data Structure

As applications are getting complex and data rich, there are three common problems that applications face now-a-days.

- **Data Search** – Consider an inventory of 1 million(10^6) items of a store. If the application is to search an item, it has to search an item in 1 million(10^6) items every time slowing down the search. As data grows, search will become slower.
- **Processor speed** – Processor speed although being very high, falls limited if the data grows to billion records.
- **Multiple requests** – As thousands of users can search data simultaneously on a web server, even the fast server fails while searching the data.

To solve the above-mentioned problems, data structures come to rescue. Data can be organized in a data structure in such a way that all items may not be required to be searched, and the required data can be searched almost instantly.

Execution Time Cases

There are three cases which are usually used to compare various data structure's execution time in a relative manner.

Worst Case – This is the scenario where a particular data structure operation takes maximum time it can take. If an operation's worst case time is $f(n)$ then this operation will not take more than $f(n)$ time where $f(n)$ represents function of n .

Average Case – This is the scenario depicting the average execution time of an operation of a data structure. If an operation takes $f(n)$ time in execution, then m operations will take $mf(n)$ time.

Best Case – This is the scenario depicting the least possible execution time of an operation of a data structure. If an operation takes $f(n)$ time in execution, then the actual operation may take time as the random number which would be maximum as $f(n)$.

Basic Terminology

Data – Data are values or set of values.

Data Item – Data item refers to single unit of values.

Group Items – Data items that are divided into sub items are called as Group Items.

Elementary Items – Data items that cannot be divided are called as Elementary Items.

Attribute and Entity – An entity is that which contains certain attributes or properties, which may be assigned values.

Entity Set – Entities of similar attributes form an entity set.

Field – Field is a single elementary unit of information representing an attribute of an entity.

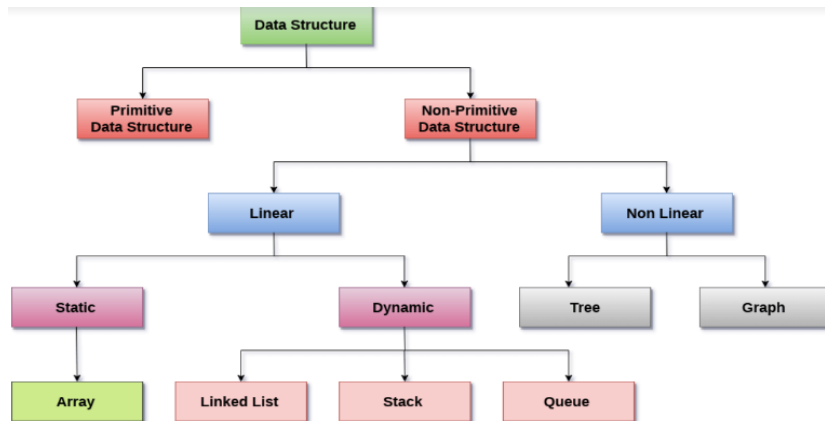
Record – Record is a collection of field values of a given entity.

File – File is a collection of records of the entities in a given entity set.

❖ Data types

- In the simplest way, Data structure can be defined as organization of data and permissible operations on data. Data structure is categorized by primitive data structure and nonprimitive data structure.

- The data structure which is directly operated upon by machine level instructions is called primitive data structure. The data structure which is not primitive is called non-primitive data structure.
- Non-primitive data structure is further categorized as linear data structure and non-linear data structure.
- In linear data structure, elements are accessed and processed in linear manner one by one which is also known as sequential data structure, Non-linear data structure is not accessed and processed in linear way but some other mechanism is required.
- The representation of data structure in memory of computer is called storage structure. The storage structure representation in an auxiliary memory is called file structure.



Primitive Data Structure

- In C programming language storage representation of int occupies 2 bytes of memory.
- For signed int first bit is used as signed bit and remaining bits are used to store data.
- In unsigned int all 16 bits are used to store data. All basic arithmetic operations can be performed on int data.
- Char data occupies one byte of memory and stores ASCII value of character.
- Real data like float and double are used to store data with fractional part. It uses IEEE format to store number with decimal point. Float 4 Bytes 1-bit sign bit 8-bits Exponential 23-bits Mantissa double 8 Bytes 1-bit sign bit 11-bits Exponential 52-bits Mantissa long double 10 Bytes 1-bit sign bit 15-bits Exponential 64-bits Mantissa.

Non-Primitive Data Structure

Linear Data Structures: A data structure is called linear if all of its elements are arranged in the linear order. In linear data structures, the elements are stored in non-hierarchical way where each element has the successors and predecessors except the first and last element.

Types of Linear Data Structures are given below:

Arrays: An array is a collection of similar type of data items and each data item is called an element of

the array. The data type of the element may be any valid data type like char, int, float or double. The elements of array share the same variable name but each one carries a different index number known as subscript. The array can be one dimensional, two dimensional or multidimensional.

The individual elements of the array age are:

age[0], age[1], age[2], age[3],..... age[98], age[99].

Linked List: Linked list is a linear data structure which is used to maintain a list in the memory. It can be seen as the collection of nodes stored at non-contiguous memory locations. Each node of the list contains a pointer to its adjacent node.

Stack: Stack is a linear list in which insertion and deletions are allowed only at one end, called top. A stack is an abstract data type (ADT), can be implemented in most of the programming languages. It is named as stack because it behaves like a real-world stack, for example: - piles of plates or deck of cards etc.

Queue: Queue is a linear list in which elements can be inserted only at one end called rear and deleted only at the other end called front. It is an abstract data structure, similar to stack. Queue is opened at both end therefore it follows First-In-First-Out (FIFO) methodology for storing the data items.

Non Linear Data Structures:

This data structure does not form a sequence i.e. each item or element is connected with two or more other items in a non-linear arrangement. The data elements are not arranged in sequential structure.

Types of Non Linear Data Structures are given below:

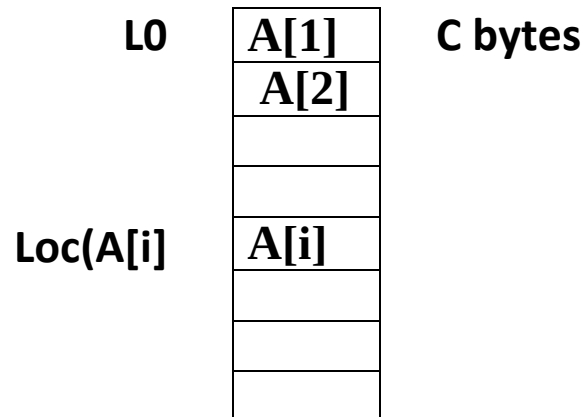
Trees: Trees are multilevel data structures with a hierarchical relationship among its elements known as nodes. The bottommost nodes in the hierarchy are called leaf node while the topmost node is called root node. Each node contains pointers to point adjacent nodes. Tree data structure is based on the parent-child relationship among the nodes. Each node in the tree can have more than one children except the leaf nodes whereas each node can have atmost one parent except the root node. Trees can be classified into many categories which will be discussed later in this tutorial.

Graphs: Graphs can be defined as the pictorial representation of the set of elements (represented by vertices) connected by the links known as edges. A graph is different from tree in the sense that a graph can have cycle while the tree cannot have the one.

❖ Array

Array is a fixed-size, sequential and homogenous collection of data. Array is the simplest data structure which makes use of computed address to locate its elements. One dimensional array is sequential allocated collection of elements which is also known as vector.

To locate any element of vector A, each element of vector occupies C bytes then address of element A[i] is computed using formula of loc(A[i]).



Storage structure of Array: $Loc(A[i]) = L_0 + C * (i - 1)$

Two Dimensional Array

Basically implementation point of view array is of following two types.

1. Row Major Array
2. Column Major Array.

1. Row Major array accesses elements by row wise. C language provides row major array. For example Array A[3, 3] in row major is allocated as follows.

A[1,1]	A[1,2]	A[1,3]
A[2,1]	A[2,2]	A[2,3]
A[3,1]	A[3,2]	A[3,3]

In Row major array elements are allocated in memory as A[1,1] A[1,2] A[1,3] A[2,1] A[2,2] A[2,3] A[3,1] A[3,2] A[3,3]

2. Column Major array accesses elements by Column wise. FORTRAN language provides column major array. For example Array A[3, 3] in column major is allocated as follows.

A[1,1]	A[2,1]	A[3,1]
A[1,2]	A[2,2]	A[3,2]
A[1,3]	A[2,3]	A[3,3]

In Column major array elements are allocated in memory as A[1,1] A[2,1] A[3,1] A[1,2] A[2,2] A[3,2] A[1,3] A[2,3] A[3,3]

Address Calculation in two dimensional array :

Location of element A[i, j] of 2D array A[m,n] where m represents total number of rows and n represents total number of columns, is calculated as

$$\text{Loc}(A[i, j]) = L_0 + (i - 1) * n + (j - 1)$$

For example Array A[3,4] has following elements

These elements are allocated in memory as sequence

A[1,1] A[1,2] A[1,3] A[1,4] A[2,1] A[2,2] A[2,3] A[2,4] A[3,1] A[3,2] A[3,3] A[3,4]

$$\text{Loc}(A[i, j]) = L_0 + (i - 1) * 4 + (j - 1)$$

For example $\text{Loc}(A[2,3]) = L_0 + 6$

$$\text{Loc}(A[3,2]) = L_0 + 9$$

3 Dimensional Array

3 Dimensional Array A[2,3,4] is allocated in row major in sequence as

A[1,1,1] A[1,1,2] A[1,1,3] A[1,1,4]

A[1,2,1] A[1,2,2] A[1,2,3] A[1,2,4]

A[1,3,1] A[1,3,2] A[1,3,3] A[1,3,4]

A[2,1,1] A[2,1,2] A[2,1,3] A[2,1,4]

A[2,2,1] A[2,2,2] A[2,2,3] A[2,2,4]

A[2,3,1] A[2,3,2] A[2,3,3] A[2,3,4]

Element allocation function $f(i,j,k) = (i-1)*12 + (j-1)*4 + (k-1)$

Applications of Array

Numbers of applications exist of array. Some of them are listed below.

1. Finding maximum, minimum and mean of elements.
2. Counting total number of elements, median and mode.
3. Arranging elements in ascending and descending order.
4. Comparing two arrays.
5. Concatenating and merging two arrays.
6. Matrix operations like addition, subtraction, multiplication, transpose, trace and inverse of matrix.
7. Storing and manipulating multi dimensional data.
8. Reverse the array.
9. Insert and delete elements from array.
10. Polynomial representation using array.

Polynomial can be represented using array.

For example $2X^2 + 5XY + Y^2$ can be represented using two dimensional array as follows.

	0	1	2	3	4
0	0	0	1	0	0
1	0	5	0	0	0
2	2	0	0	0	0
3	0	0	0	0	0
4	0	0	0	0	0

$X^2 + 3XY + Y^2 - Y + X$ can be represented using two dimensional array as follows.

	0	1	2	3	4
0	0	-1	1	0	0
1	1	3	0	0	0
2	1	0	0	0	0
3	0	0	0	0	0
4	0	0	0	0	0

$X^4 - 2X^3Y + 3X^2Y^2 - 4XY^3 + 5Y^4$ can be represented using two dimensional array as follows.

	0	1	2	3	4
0	0	0	0	0	5
1	0	0	0	-4	0
2	0	0	3	0	0
3	0	-2	0	0	0
4	1	0	0	0	0

Sparse matrix

If a matrix has many of its element as zero then such a matrix is called sparse matrix

Sparse matrix					
	0	1	2	3	4
0	0	0	2	0	1
1	6	0	0	7	0
2	0	0	0	0	0
3	0	7	0	9	0
4	1	0	0	0	2

Pointer

Pointer is a variable which stores the address of other variables. Pointer variable is declared using `*`.

Syntax : Datatype *Pointer;

For example following declarative statement declared an integer pointer which can store address of any integer variable.

```
int *p;
```

Initialization of pointer can be made at the time of declaring pointer as initialized in following statement.

```
int a;
```

```
int *p = &a;
```

Variable a must be declared prior to the initialize.

Dereferencing a pointer meaning accessing a value stored in variable via pointer. The operator `'*'` is an indirection operator is used to dereference the pointer.

For example if

```
int a = 10; int *p = &a;
```

then `printf("%d ", *p);` will display 10.

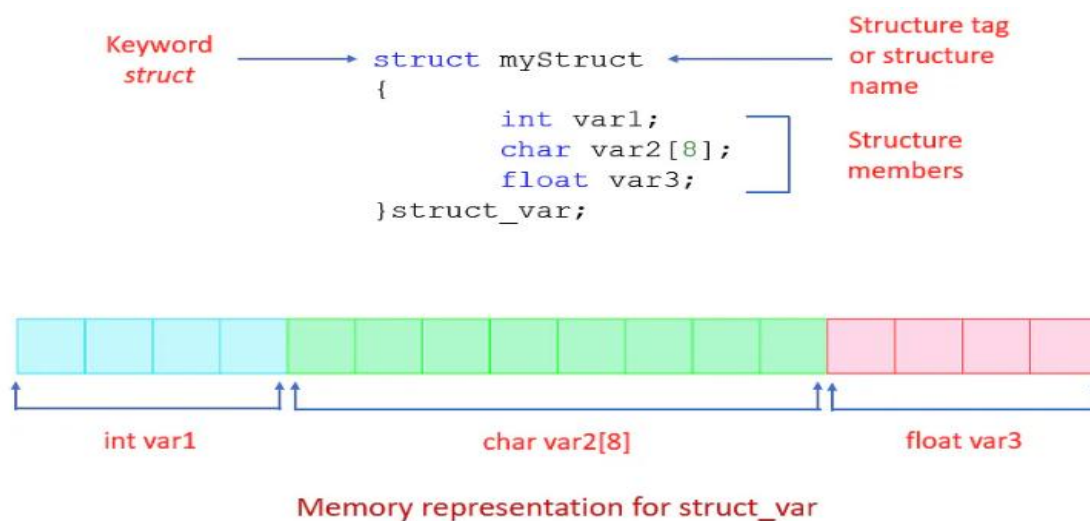
Pointer Arithmetic

Some arithmetic operations can be performed on pointer like adding integer number, subtracting an integer number and subtraction of two pointers.

Pointer arithmetic actually not a simple arithmetic but after addition or subtraction of an integer, it just point to the next element of specified data type. If p is an integer pointer then p++ will add size of integer in bytes, if p is a float pointer then p++ will add size of float in bytes so that it can point to the next element.

Structure

A structure is a keyword that creates user-defined data types in C/C++. A structure creates a data type that can be used to group items of possibly different types into a single type



A structure can be defined as a single entity holding variables of different data types that are logically related to each other. All the data members inside a structure are accessible to the functions defined outside the structure. To access the data members in the main function, you need to create a structure variable. Structure in C programming is very helpful in cases where we need to store similar data of multiple entities.

```
#include<stdio.h>  
#include<conio.h>  
void main( )  
{  
    struct employee  
{
```

```

int id ;
float salary ;
int mobile ;
};
struct employee e1,e2,e3 ;
clrscr();
printf ("\nEnter ids, salary & mobile no. of 3 employee\n");
scanf ("%d %f %d", &e1.id, &e1.salary, &e1.mobile);
scanf ("%d%f %d", &e2.id, &e2.salary, &e2.mobile);
scanf ("%d %f %d", &e3.id, &e3.salary, &e3.mobile);
printf ("\n Entered Result ");
printf ("\n%d %f %d", e1.id, e1.salary, e1.mobile);
printf ("\n%d%f %d", e2.id, e2.salary, e2.mobile);
printf ("\n%d %f %d", e3.id, e3.salary, e3.mobile);
getch();
}

```

Sorting Techniques

Sorting allows an efficient arrangement of elements within a given data structure. It is a way in which the elements are organized systematically for some purpose. For example, a dictionary in which words are arranged in alphabetical order and telephone director in which the subscriber names are listed in alphabetical order. There are many sorting techniques out of which we study the following.

- Selection Sort
- Bubble Sort
- Quick Sort
- Merge Sort
- Insertion Sort

Selection Sort

- Selection sort is one of the simplest sorting technique.
- In first Pass, first element is interchanged with the selected smallest element. It continues elements for second, third upto N-1 passes for sorting N elements array
- If no smallest element is found, no interchange takes place.
- At each iteration this technique finds the smallest element and places it to its proper place that's why it is known as selection sort.

Procedure SELECTION_SORT(K, N). Given a vector K of N elements, this procedure rearrange the vector in ascending order, that is, its elements will be in the order $K[1] \leq K[2] \leq \dots \leq K[N]$. The sorting process is based on the techniques just described. The variable PASS denotes the pass index and the position of the first element in the vector which is to be examined during a particular pass. The variable MIN_INDEX denotes the position of the smallest element

encountered thus far in a particular pass. The variable I is used to index elements K[PASS] to K[N] in a given pass. All variables are of type integer.

1. [Loop on pass index]
Repeat thru step 4 for PASS = 1, 2, 3, ... , N -1.
2. [Initialize minimum index]
MIN_INDEX $\leftarrow$ PASS
3. [Make a pass and obtain element with smallest value]
Repeat for I = PASS + 1, PASS + 2, ..., N
If $K[I] < K[\text{MIN_INDEX}]$
then MIN_INDEX $\leftarrow$ I
4. [Exchange elements]
If MIN_INDEX $\neq$ PASS
then $K[\text{PASS}] \longleftrightarrow K[\text{MIN_INDEX}]$
5. [Finished]
Return.

Following is the program for selection sort

```
// Program for Selection Sort
#include<conio.h>
#include<stdio.h>
void main()
{
    int K[100], MIN_INDEX, PASS, I, N, TMP;
    printf("N =");
    scanf("%d", &N);
    // Loop on PASS index */
    for (I = 0; I < N; I++)
    {
        printf("K[%d] = ", I);
        scanf("%d", &K[I]);
    }
    for (PASS = 0; PASS < N - 1; PASS++)
    {
        MIN_INDEX = PASS;
        for (I = PASS+1; I < N; I++)
            if (K[I] < K[MIN_INDEX])
                MIN_INDEX = I;
        if(MIN_INDEX != PASS)
        {
            TMP = K[MIN_INDEX];
```

```

K[MIN_INDEX] = K[PASS];
K[PASS] = TMP;
}
for (l = 0; l < N; l++)
printf("%d ", K[l]);
printf("\n");
}
for (l = 0; l < N; l++)
printf("%d ", K[l]);
printf("\n");
}

```

Output

Total Elements = 10

K[0] = 42

K[1] = 23

K[2] = 74

K[3] = 11

K[4] = 65

K[5] = 58

K[6] = 94

K[7] = 36

K[8] = 99

K[9] = 87

11 23 74 42 65 58 94 36 99 87

11 23 74 42 65 58 94 36 99 87

11 23 36 42 65 58 94 74 99 87

11 23 36 42 65 58 94 74 99 87

11 23 36 42 58 65 94 74 99 87

11 23 36 42 58 65 94 74 99 87

11 23 36 42 58 65 74 94 99 87

11 23 36 42 58 65 74 87 99 94

11 23 36 42 58 65 74 87 94 99

11 23 36 42 58 65 74 87 94 99

BUBBLE SORT:

- Bubble sort is another well-known sorting technique.
- Instead of finding smallest element and performing exchange in selection sort, bubble sort exchanges two immediate elements if it discovers that they are out of order.
- Bubble sort iteration reduces the size of unsorted list by one at each iteration.

- It keeps track of number of exchanges are made, If in any pass if there is no exchange of elements then bubble sort algorithm is terminated because in sorted list there is no need of exchanging elements.
- It requires maximum N-1 passes to sort the element.

Procedure BUBBLE_SORT(K, N). Given a vector K of N elements, this procedure sorts the elements into ascending (increasing) order using the method just described. The variables PASS and LAST denote the pass counter and position of the last unsorted element, respectively. The variable I is used to index the vector elements. The variables EXCHS is used to count the number of exchanges made on any pass. All variables are integer.

1. [Initialize]
 - LAST $\leftarrow$ N (entire list assumed unsorted at this point)
2. [Loop on pass index]
 - Repeat thru step 5 for PASS = 1, 2, ..., N-1
3. [Initialize exchanges counter for this pass]
 - EXCHS $\leftarrow$ 0
4. [Perform pair-wise comparisons on unsorted elements]
 - Repeat for I = 1, 2, ..., LAST - 1
 - If $K[I] > K[I + 1]$
 - then $K[I] \longleftrightarrow K[I + 1]$
 - EXCHS $\leftarrow$ EXCHS + 1
5. [Were any exchanges made on this pass]
 - If EXCHS = 0
 - then Return (mission accomplished; return early)
 - else LAST $\leftarrow$ LAST - 1 (reduce size of unsorted list)
6. [Finished]
 - Return (maximum number of passes required)

Program for Bubble Sort

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int K[100], I, N, PASS, EXCHS, LAST, TMP;
    clrscr();
    printf("N = ");
    scanf("%d", &N);
    // Loop on PASS index
    for (I = 0; I < N; I++)
    {
        printf("K[%d] = ", I);
        scanf("%d", &K[I]);
    }
    printf("Before Sorting\n");
    for (I = 0; I < N; I++)
```

```

    printf("%d ", K[I]);

printf("\n\n");
LAST = N - 1;

for (PASS = 0; PASS < N - 1; PASS++)
{
    EXCHS = 0;
    for (I = 0; I < LAST; I++)
    {
        if (K[I] > K[I+1])
        {
            TMP = K[I];
            K[I] = K[I+1];
            K[I+1] = TMP;
            EXCHS++;
        }
    }
    if (EXCHS == 0)
        break;
    else
        LAST--;
    for (I = 0; I < N; I++)
        printf("%d ", K[I]);
    printf("\n");
}
printf("\n\nAfter Sorting\n");
for (I = 0; I < N; I++)
    printf("%d ", K[I]);
printf("\n");
}

```

Output

```

N = 10
K[0] = 42
K[1] = 23
K[2] = 74
K[3] = 11
K[4] = 65
K[5] = 58
K[6] = 94
K[7] = 36
K[8] = 99
K[9] = 87
Before Sorting
42 23 74 11 65 58 94 36 99 87
23 42 11 65 58 74 36 94 87 99
23 11 42 58 65 36 74 87 94 99
11 23 42 58 36 65 74 87 94 99

```

11 23 42 36 58 65 74 87 94 99

11 23 36 42 58 65 74 87 94 99

After Sorting

11 23 36 42 58 65 74 87 94 99

QUICK SORT:

QUICK SORT:

Procedure QUICK_SORT(K, LB, UB). Given a table K of N records, this recursive procedure sorts the table, as previously described, in ascending order. A dummy record with key $K[N + 1]$ is assumed where $K[I] \leq K[N + 1]$ for all $1 \leq I \leq N$. The integer parameters LB and UB denote the lower and upper bounds of the current subtable being processed. The indices I and J are used to select certain keys during the processing of each subtable. KEY contains the key value which is being placed in its final position within the sorted subtable. FLAG is a logical variable which indicates the end of the process the places a record in its final position. When FLAG becomes false, the input subtable has been partitioned into two disjointed parts.

1. [Initialize]
 FLAG $\leftarrow$ true
2. [Perform sort]
 If LB < UB
 then I $\leftarrow$ LB
 J $\leftarrow$ UB + 1
 KEY $\leftarrow$ K[LB]
 Repeat while FLAG
 I $\leftarrow$ I + 1
 Repeat while K[I] < KEY (scan the keys from left to right)
 I $\leftarrow$ I + 1
 J $\leftarrow$ J - 1
 Repeat while K[J] > KEY (scan the keys from right to left)
 J $\leftarrow$ J - 1
 If I < J
 then K[I] $\leftrightarrow$ K[J] (interchange records)
 else FLAG $\leftarrow$ false
 K[LB] $\leftrightarrow$ K[J] (interchange records)
 Call QUICK_SORT(K, LB, J - 1) (sort first subtable)
 Call QUICK_SORT(K, J + 1, UB) (sort second subtable)
3. [Finished]
 Return

MERGE SORT:

- ✓ Merge sort employs divide and data.
- ✓ Merge sort divides and conquer technique to sort the the entire sorting problem into small number of sub problems.
- ✓ After getting solution (sorted data) of sub problems it merges the solutions of subproblems to obtain solution of original problem.
- ✓ Conquer means solving small part of problem.
- ✓ Merge sort actually involves following three steps.
 - (1) Divide
 - (2) Conquer
 - (3) Combine

Procedure SIMPLE_MERGE(K, FIRST, SECOND, THIRD). Given two ordered subtables stored in a vector K with FIRST, SECOND and THIRD as just described, this procedure performs a simple merge. TEMP is a temporary vector used in the merging process. The variables I and J denote the cursor associated with the first and second subtables, respectively. L is an index variable associated with the vector TEMP.

1. [Initialize]
 - $I \leftarrow \text{FIRST}$
 - $J \leftarrow \text{SECOND}$
 - $L \leftarrow 0$
2. [Compare corresponding elements and output the smallest]
 - Repeat while $I < \text{SECOND}$ and $J \leq \text{THIRD}$
 - If $K[I] \leq K[J]$
 - then $L \leftarrow L + 1$
 - $\text{TEMP}[L] \leftarrow K[I]$
 - $I \leftarrow I + 1$
 - else $L \leftarrow L + 1$
 - $\text{TEMP}[L] \leftarrow K[J]$
 - $J \leftarrow J + 1$
3. [Copy the remaining unprocessed elements in output area]
 - If $I \geq \text{SECOND}$
 - then Repeat while $J \leq \text{THIRD}$
 - $L \leftarrow L + 1$
 - $\text{TEMP}[L] \leftarrow K[J]$
 - $J \leftarrow J + 1$
 - else Repeat while $I \leq \text{SECOND}$
 - $L \leftarrow L + 1$
 - $\text{TEMP}[L] \leftarrow K[I]$
 - $I \leftarrow I + 1$
4. [Copy elements in temporary vector into original area]
 - Repeat for $I = 1, 2, \dots, L$
 - $K[\text{FIRST} - 1 + I] \leftarrow \text{TEMP}[I]$
5. [Finished]
 - Return.

Procedure TWO_WAY_MERGE_SORT_R(K, START, FINISH). Given a vector K it is required to sort recursively its elements between positions START and FINISH (inclusive). SIZE denotes the number of elements in the current subtable to be sorted. MIDDLE denotes the position of the middle element of that subtable.

1. [Compute the size of the current subtable]
 $SIZE \leftarrow FINISH - START + 1$
2. [Test base condition for subtable of size one]
 If $SIZE \leq 1$
 then Return
3. [Calculate midpoint position of current subtable]
 $MIDDLE \leftarrow START + \lceil SIZE / 2 \rceil - 1$
4. [Recursively sort first subtable]
 Call TWO_WAY_MERGE_SORT_R(K, START, MIDDLE)
5. [Recursively sort second subtable]
 Call TWO_WAY_MERGE_SORT_R(K, MIDDLE + 1, FINISH)
6. [Merge two ordered subtables]
 Call SIMPLE_MERGE(K, START, MIDDLE + 1, FINISH)
7. [Finished]
 Return.

Procedure MERGE_PASS(R, N, C, L). Given a table R of N records which is considered to be partitioned into ordered subtables, each of which contains L records (or fewer), this algorithm merges these subtables by pairs. The records of R are denoted by $R[1]$, $R[2]$, ..., $R[N]$. An auxiliary table C is required in the merging process. The variables P and Q keep track of which pair of subtables are currently being merged. Since N is not necessarily an integral power of 2, the sizes of the subtables being merged are not always the same (i.e., L). The sizes of the tables are given by variables N1 and N2. The variables I and J are indices to the corresponding records of the subtables that are to be merged. The index variable S references a record in the output area, and T is a temporary index variable.

1. [Initialize first pass]
 $P \leftarrow 1$
 $N1 \leftarrow N2 \leftarrow L$
 $Q \leftarrow P + L$
2. [Perform one pass]
 Repeat thru step 7 while $Q \leq N$
3. [Initialize simple merge]
 $I \leftarrow P$
 $J \leftarrow Q$
 $S \leftarrow P$
4. [Compare corresponding records and output smallest]
 Repeat while $I - P < n1$ and $J - Q < N2$
 If $K[I] \leq K[J]$
 then $C[S] \leftarrow R[I]$

- ```

 I ← I + 1
 S ← S + 1
 else C[S] ← R[J]
 J ← J + 1
 S ← S + 1
5. [Copy the remaining unprocessed records from a subtable into output area]
 If I – P ≥ N1
 then Repeat for T = J, J + 1, ..., Q + N2 – 1
 C[S] ← R[T]
 S ← S + 1
 else Repeat for T = I, I + 1, ..., P + N1 – 1
 C[S] ← R[T]
 S ← S + 1
6. [Update and test direct subtable index]
 P ← Q + N2
 If P > N
 then Return
7. [Update Q and check bound of second subtable]
 Q ← P + L
 If Q + L > N + 1
 then N2 ← N – Q + 1
8. [Copy unmatched subtable]
 Repeat for T = P, P + 1, ..., N
 C[T] ← R[T]
9. [Finished]
 Return.

```

## **Insertion Sort**

### **Searching**

Searching is one of the most important and popular application in computer science.

Searching means searching a record for a given key value.

Two widely used searching techniques are

- (1) Linear Search (Sequential Search)
- (2) Binary Search

### **Sequential Searching**

In linear search a data structure is searched for a given key value one by one in linear fashion. If element is found it is returned otherwise search will be failed. Time require for linear search is proposal to total number of elements N and it is denoted as O(N).

**Function Linear\_SEARCH(K, N, X).** Given an unordered vector K consisting of N + 1 (N >= 1) elements, this algorithm searches the vector for a particular element having the value of X. Vector element K[N + 1] serves as a sentinel element and receives the value

of X prior to the search. The function returns the index of the vector element if the search is successful, and returns 0 otherwise.

1. [Initialize search]  
     $I \leftarrow 1$   
     $K[N + 1] \leftarrow X$
2. [Search the vector]  
    Repeat while  $K[I] \neq X$   
         $I \leftarrow I + 1$
3. [Successful search?]  
    If  $I = N + 1$   
    then Write('UNSUCCESSFUL SEARCH')  
        Return(0)  
    else Write('SUCCESSFUL SEARCH')  
        Return(I)

## **Binary Searching**

- ✓ Binary search is one of the most important and popular searching technique.
- ✓ At each iteration, It divides entire list to half of the current list.
- ✓ Prime condition to apply binary search is that data stored in list must be in sorted order. For linear search it is not necessary
- ✓ First it tests the mid element of list. If it is not the search element then lower or higher limit is changed accordingly.
- ✓ Time require for binary search is  $O(\log n)$ .

## **Example**

**Function BINARY\_SEARCH(K, N, X).** Given a Vector K, consisting of N elements in ascending order, this algorithm searches the structure for a given element whose value is given by X. The variables LOW, MIDDLE, and HIGH denote the lower, middle, and upper limits of the search interval, respectively. The function returns the index of the vector element if the search is successful, and returns 0 otherwise.

1. [Initialize]  
     $LOW \leftarrow 1$   
     $HIGH \leftarrow N$
2. [Perform search]  
    Repeat thru step 4 while  $LOW \leq HIGH$
3. [Obtain index of midpoint of interval]  
     $MIDDLE \leftarrow [(LOW + HIGH) / 2]$
4. [Compare]  
    If  $X < K[MIDDLE]$   
    Then  $HIGH \leftarrow MIDDLE - 1$   
    Else If  $X > K[MIDDLE]$

```
Then LOW $\leftarrow$ MIDDLE + 1
Else Write('SUCCESSFUL SEARCH')
 Return(MIDDLE)
```

5. [Unsuccessful search]  
 Write('UNSUCCESSFUL SEARCH')  
 Return(0)